

HW 11

Sam Ly

April 24, 2026

Chapter 22

1. Come up with something more efficient. Do you think the additional complexity is worth it?

We can use a hashmap to map the variables to their stack index in constant time, however this is not worth it as this cost is only incurred at compile time.

I would rather have the compiler be simple, and worry about performance when we need to.

2. What would you do if it was your language? Why?

Other languages tend to make code like this a compile error. I believe this makes sense, as this is completely ambiguous behavior. One language that I see that allows this is Rust, however Rust semantics allow for this to make sense throw something called ‘shadowing’.

3. Pick a keyword for a single-assignment variable form to add to Lox. Justify your choice, then implement it. An attempt to assign to a variable declared using your new keyword should cause a compile error.

I would add a keyword ‘const’ that marks a variable as constant. The implementation is too long to fit here, but the overall idea is to add an extra check for the assignment operator, that checks if the left hand side has been marked const.

4. Extend clox to allow more than 256 local variables to be in scope at a time.

See code on github.

Chapter 23

1. To keep things simpler, we’re omitting fallthrough and break statements. Each case automatically jumps to the end of the switch statement after its statements are done.

This can be accomplished through desugaring, since we do not need the fallthrough logic. Switch statements can essentially be converted to if statements.

2. Make sure to think about scope. What should happen to local variables declared inside the body of the loop or in blocks nested inside the loop when a continue is executed?

When a continue is executed, we need to “reset” the stack to the point at which the loop starts. This way, we don’t have access to garbage data, and no ‘stack leaks’ occur.

3. For fun, try to invent a useful novel control flow feature for Lox. It can be a refinement of an existing form or something entirely new. In practice, it’s hard to come up with something useful enough at this low expressiveness level to outweigh the cost of forcing a user to learn an unfamiliar notation and behavior, but it’s a good chance to practice your design skills.

One unique control flow that I have always liked was the ‘do while’ loops from C. Implementing is relatively simple, since this can just be desugared into a normal while loop.